第五章 从分析到设计

2018年10月9日 8:28

- 本章内容旨在丰富前面章节所学UML建模的语义的细节

1. 高级(advanced)类建模
   - 附加的建模概念：构造型、约束、导出信息、可见性、限定关联、关联类、参数化类。这些概念是可选的，很多模型没有它们也足够了

   a. 扩展机制，并非UML本身内容，如构造型、约束、标签定义和标签值
      - 普通的UML标准可能没有满足某些更特殊的建模需要

      i. 构造型stereotypes
         1) 构造型对现有UML元素进行扩展，使现有元素的语义多样化
         2) 表示方法，用双尖角括号括住的一个名字，如<<global>>,<<include>>
         3) 构造型扩展的是语义而不是UML结构

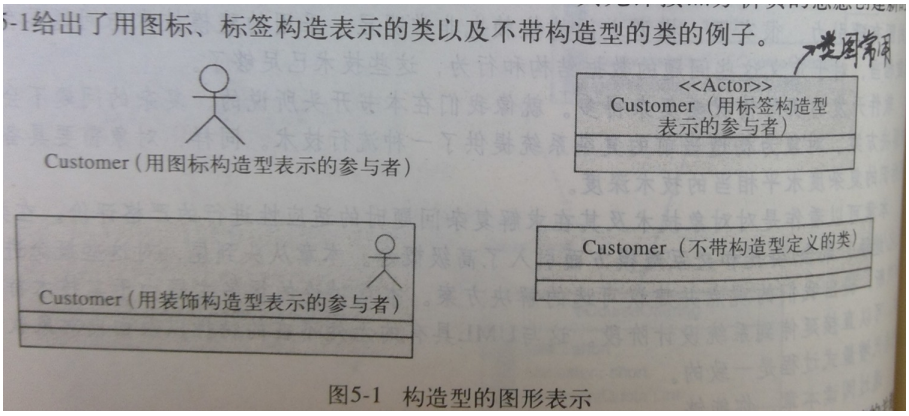


图5-1 构造型的图形表示

      ii. 注释和约束Comments and constraints
         1) 在UML任何地方都可以插入注释
         2) 注释：能表示为UML笔记，在需要注释的对象加虚线连接一个卷角书页
         3) 约束：指条件限制，在需要限制的单个对象（该对象旁添加大括号）或对象之间（关联，一个虚线依赖箭头，约束绑定在该箭头上，如iii.标签中的例子）

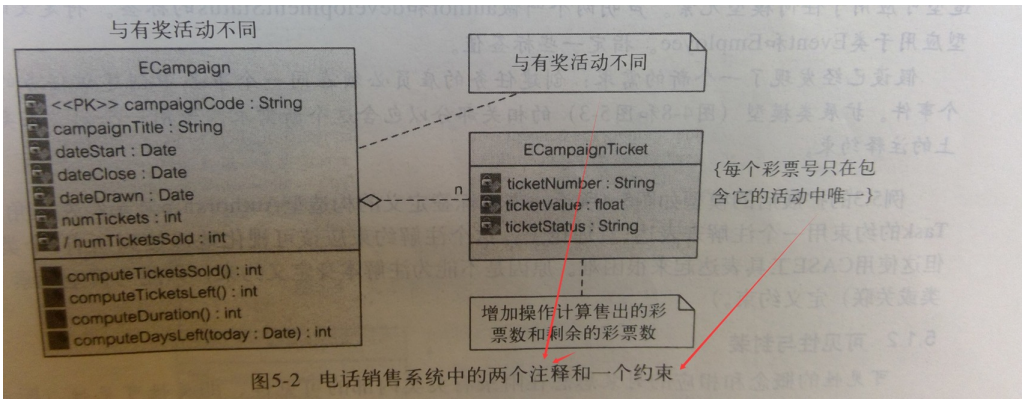


图5-2 电话销售系统中的两个注释和一个约束

      iii. 标签tags
         1) 标签的定义：是构造型的特性，表示为构造型声明的类矩形的一个属性
         2) 标签的值：是一个名-值对，附属于一个模型元素
         3) 标签与约束相似，表示模型中的任意文本信息，写在大括号中
            a) 形式如下：tag=value
            b) 如：

c)

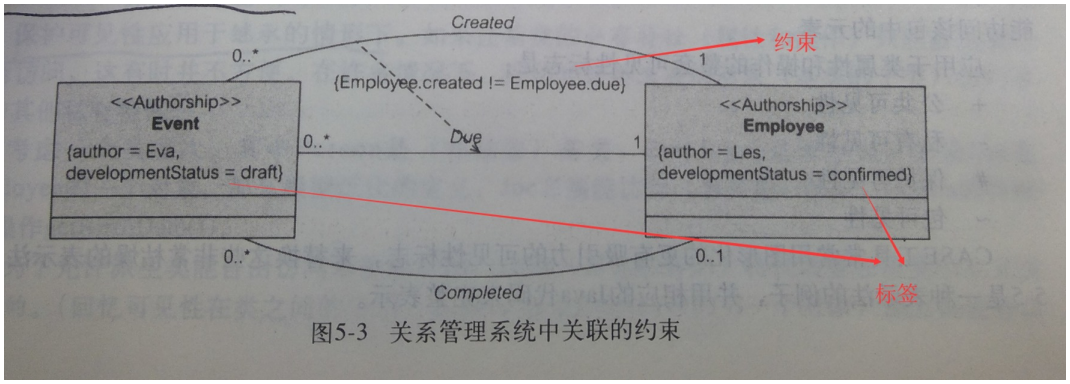


图5-3 关系管理系统中关联的约束

b. 可见性与封装

i. 可见性

□ 应用于类属性和操作的整套可见性标志：

- \+ 公共可见性
- \- 私有可见性
- # 保护可见性
- ~ 包可见性

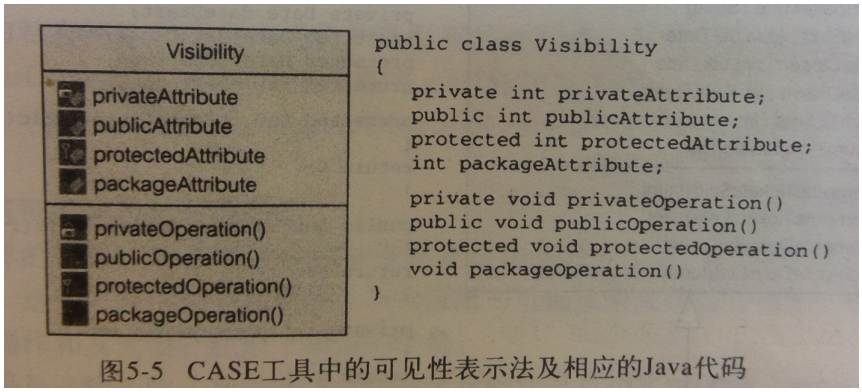


图5-5 CASE工具中的可见性表示法及相应的Java代码

1) 受保护的可见性（protected visibility），应用于继承的情况下，包含了包可见性

a) 父类的属性可以被继承他的子类访问（父类的私有特性也可以）

b)

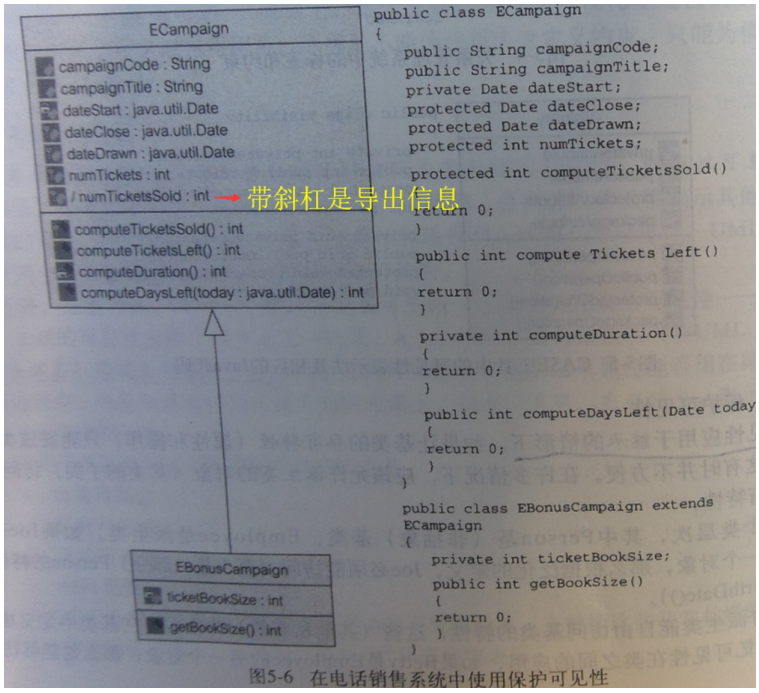


图5-6 在电话销售系统中使用保护可见性

2) java包可见性（package visibility），默认情况

a) 对一个包中的所有其他类可见

- 保护的（和公共的）也具有包访问性，但反过来不成立。即同一包中的其他类能够访问保护特性，但如果派生类和基类在不同的包中，派生类就不能访问那些具有包可见性的特

性。

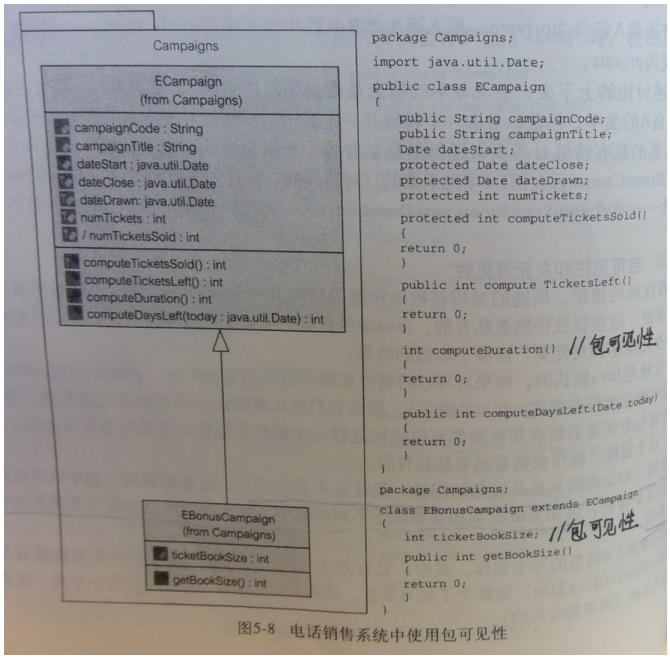
- 

图5-8 电话销售系统中使用包可见性

3) c++友元可见性Friend，类似于java的包可见性
   a) 如：friend void Book::putOnBookShelf()
   b) 友元可以是另一个类或另一个类的操作。友元关系是单向的，A类让B类成为友元，但B类不一定是A类的友元，即B类不承认
   c) 友元关系要在授予友元关系的类中声明
   d) 在UML中，友元关系显示为一条虚线表示的依赖关系，从一个友元类或操作指向授予友元关系的类。构造型<<friend>>绑定在依赖箭头上

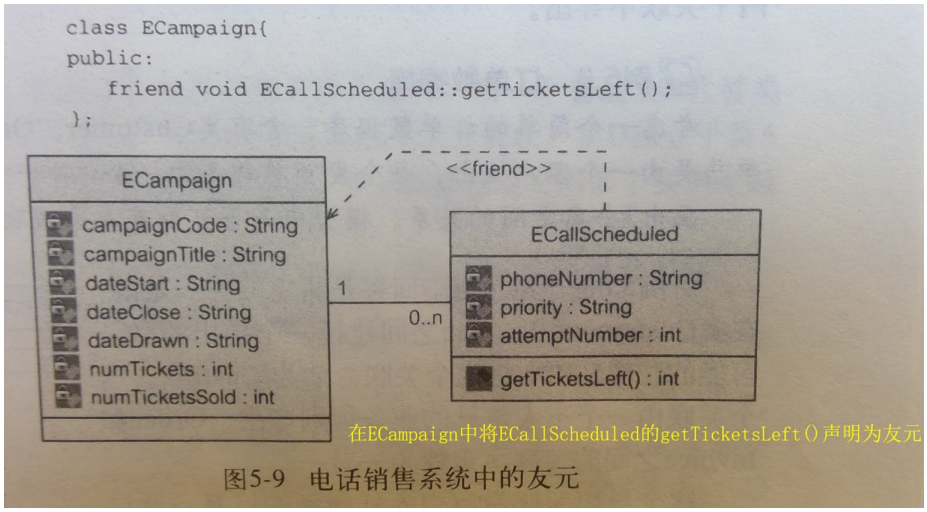
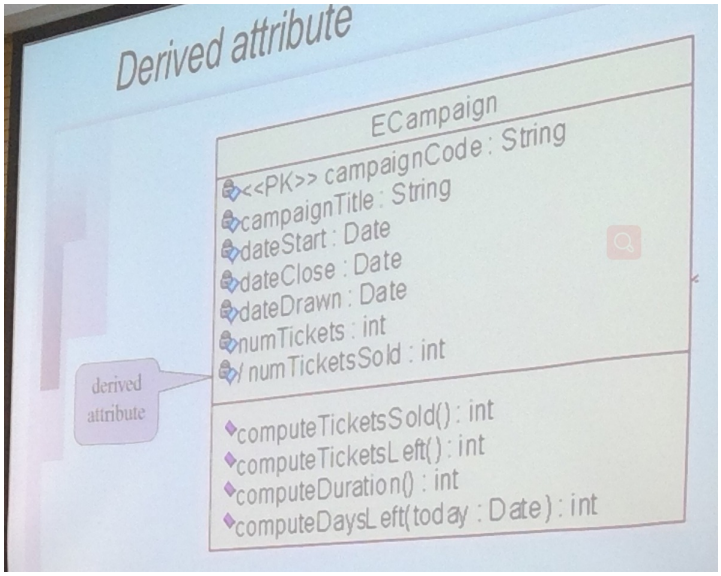


图5-9 电话销售系统中的友元

4) Public，全天下都可以访问,包含了包可见性
5) private，只能由类的内部对象访问

c. 导出信息derived information
   i. 是一种经常应用于属性或关联的约束
   ii. 导出信息在模型中是冗余的——它可以在需要时计算出来
   iii. 没有丰富分析模型的语义，但可以使模型更容易可读
   iv. 在设计模型中，需要考虑信息存取的最优化问题，所以了解哪些信息是导出的更为重要
   v. 在UML表示法是在导出属性名或关联名的前面加一条斜线"/"
      1) 导出属性



2) 导出关联，并不推荐，因为导出后形成了关联的回路

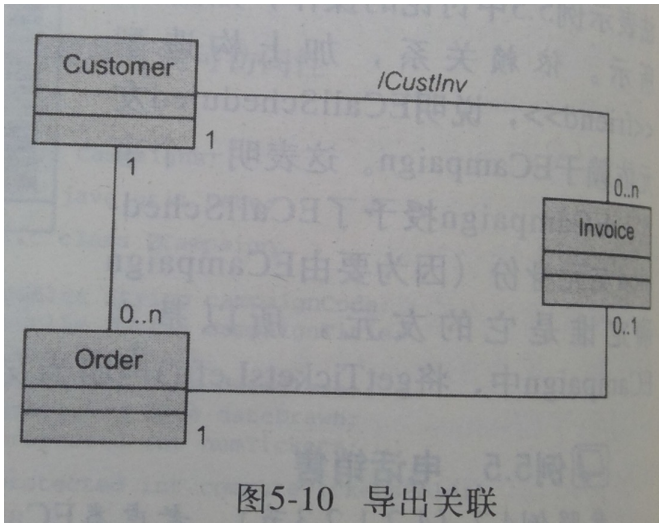


图5-10 导出关联

d. 限定关联，在关联上加属性，在二元关联的“一对多”的“多”的一端加
   i. 不用限定关联，也可以构造出完整而又充分表现力的类模型。但如果要使用限定关联，就要全面一致的使用
   ii. 通过特定的方式将一对多降成一对一，多对多降成多对一

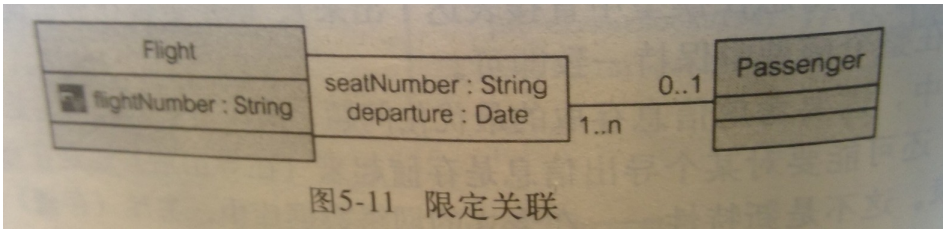


图5-11 限定关联

e. 关联类与具体化类
   i. 关联类更像ER图的多对多的联系

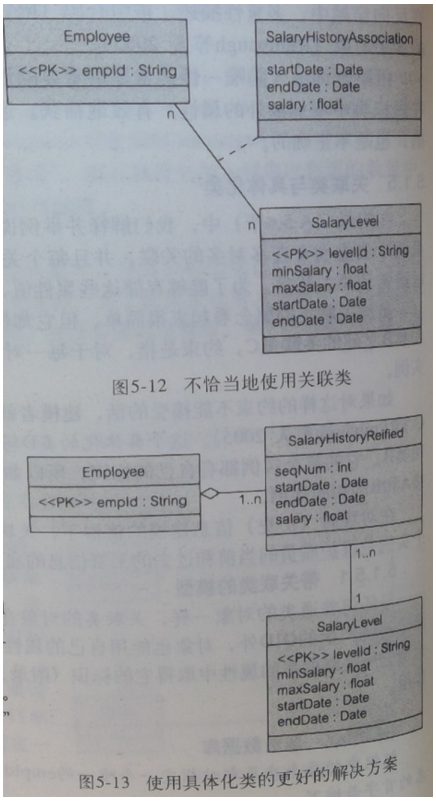


图5-12 不恰当地使用关联类

图5-13 使用具体化类的更好的解决方案

2. 高级泛化(generalization)与继承(inheritance)建模
    - 泛化是一种静态状态，是类之间的语义关系；继承是一种动作机制
    - 泛化的好处是减少关联的对象，
    a. 泛化和可替换性
        i. 虽然泛化引入了新的类，但它可以减少模型中的关联关系和聚合关系的总数
        ii. 良好运用的泛化能够提高系统模型的表现力、可理解性和抽象程度
        iii. 泛化的作用来源于可替换性原则——在代码中任何访问超类对象的地方，都可以用子类对象来替换超类对象
    b. 继承和封装，两者是互相矛盾正交关系
        i. 继承：子类可以去访问父类的属性和方法，允许子类直接访问保护属性
        ii. 封装：基于类的封装，类的内部对象的属性不能隐藏
    c. 接口继承
        - 当以可替换性为目标来使用泛化时，它就同接口继承是同义词了
        - 接口继承的别名是**子类型**
        i. 继承包括泛化和接口继承
        ii. 抽象类，只要有部分抽象方法就是抽象类
        iii. 接口里只能有常量，全是抽象方法

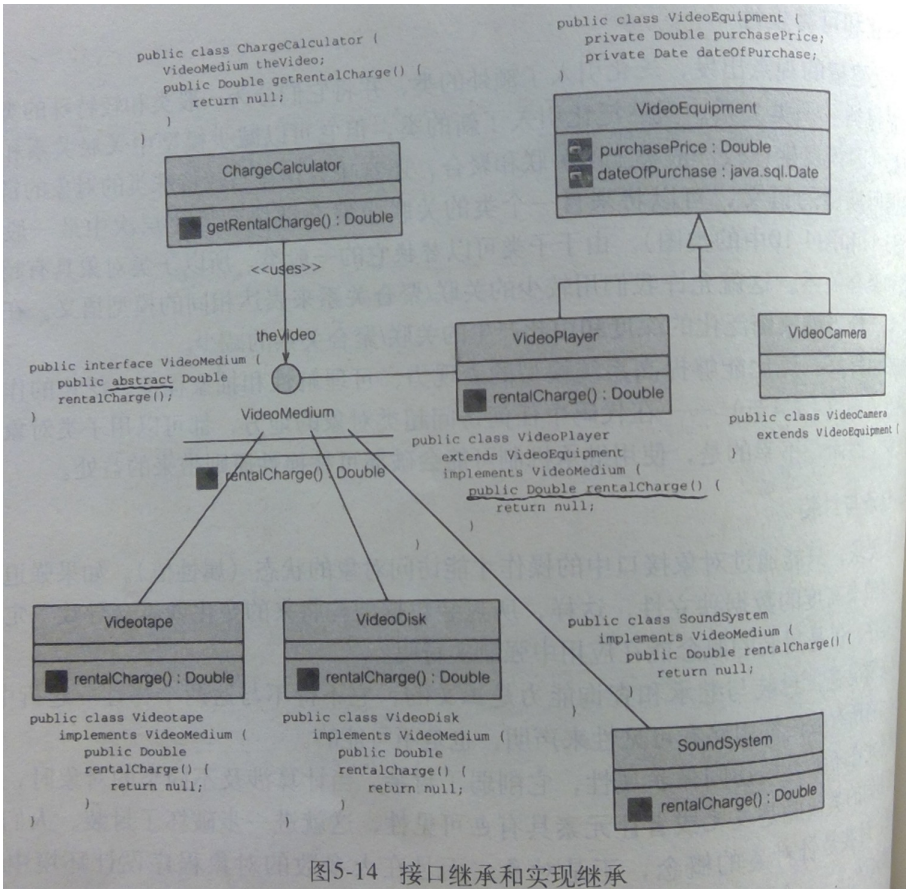


图5-14 接口继承和实现继承

d. 实现继承

i. 恰当使用——扩展继承

1) 将继承作为类的增量式定义，子类具有比超类更多的特性（属性、方法）

2)

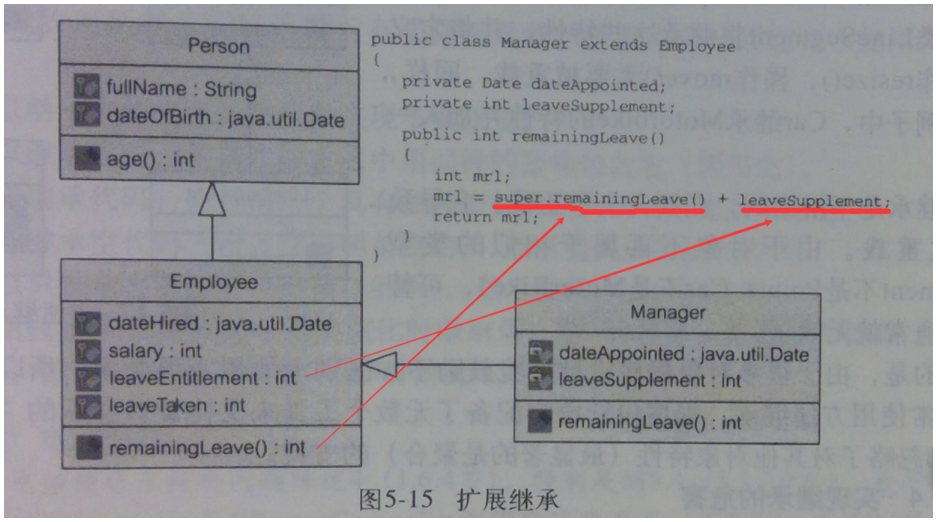


图5-15 扩展继承

ii. 有问题的使用——限制继承

1) 有一些继承来的特性在子类中被禁止（重载）

2) **子类中一些继承来的特性被覆盖**，叫限制继承

iii. 不恰当的使用——方便继承

iv. 实现继承的危害：

- 自递归总是发生在**实现继承**
- 实现继承按很多标准来说都是一件有风险的事情

1) 脆弱基类：父类属性方法的改变会影响子类，

a) 只有尽可能将子类的共有属性抽取出来

2) 重载和回调（向上调用、向下调用）

a) 重载，

就是函数或者方法有相同的名称，但是参数列表不相同的情形，这样的同名不同参数的函数或者方法之间，互相称之为重载函数或者方法。

向上造型：父类引用指向子类对象

注:调用时看引用类型,并不能调用子类的,变量和子类没有涉及重写的方法,

向下造型：子类引用指向父类对象

b)

```
public class test {
        public static void main(String[] args) {
                Aoo aoo=new Boo();//向上造型自动类型转换
                Boo o1=(Boo)aoo;//引用aoo所指向的对象，强转为Boo类型
                Inter1 o2=o1;//引用o1所指向的对象
                if(aoo instanceof Coo) {//强转时用instanceof判断，避免出错
                        Coo o3=(Coo)aoo;//false
                }
        }
}
interface Inter1{

}
class Aoo{

}
class Boo extends Aoo implements Inter1{

}
class Coo extends Aoo{

}
```

3) 多重继承
   a) 多重接口继承，考虑了接口协议的合并
   b) 多重实现继承，允许实现片段的合并，可用接口继承替代

3. 高级聚合与委托(delegation)建模
   - 一个复合类包含一个或多个构件类，构件类是其复合类的元素

   a. 给聚合增加更多的语义
      i. exclusiveOwns聚合（复合）
         1) {frozen}约束应用于构件类，它说明一个构件类对象不能再连接到另一个复合对象上。构建对象可以被完全删除，但不能将它转换给另一个所有者。

2)

图5-19给出了两个ExclusiveOwns聚合的例子，左边这个例子用UML值语义（一个实心菱形）建模，右边这个例子用UML引用语义（一个空心菱形）建模。

一个Chapter对象最多属于一个CopyrightedBook的一部分。一旦它（通过值）被结合到一个CopyrightedBook对象中，就不能再被连接到另一个组合对象中。这个连接被冻结了。

一个BridgeCardPack精确包含52张牌，我们使用UML引用语义来建模这个所属关系。任何一个BridgeCard对象都正好属于一个BridgeCardPack，并且不能再被连接到另一副牌中。

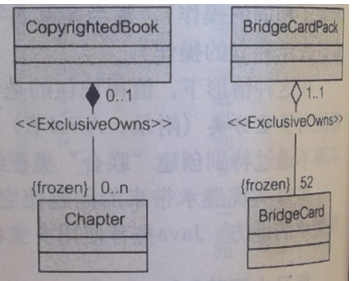


图5-19 ExclusiveOwns聚合

      ii. Owns聚合（复合）
         1) 一个构建对象最多属于一个复合对象，但它能被再连接到一个复合对象上。当一个复合对象被删除，它的构件对象也被删除

2)

如同ExclusiveOwns聚合一样，在UML中，可以用复合的值语义（一个实心菱形）或者用聚合的引用语义（一个空心菱形）来表达Owns聚合。在任何时刻，一个构件对象最多属于一个复合对象，但它能够被再连接到另一个复合对象上。当一个复合对象被删除时，它的构件对象也被删除。

图5-20展示了Owns聚合的两个例子，一个Water对象能够从一个Jug再连接到另一个Jug上。同样，一个Tire对象能够从一辆Bicycle调换到另一辆Bicycle上。由于存在

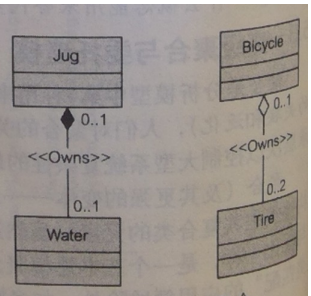


图5-20 Owns聚合

iii. Has聚合

1) 不具有存在依赖性，复合对象的删除不会自动向下扩散到构件对象上

2) 只有传递性和非对称性的特点

3)

5.3.1.3 Has聚合

在UML中，通常用聚合的引用语义（一个空心菱形）来建模Has聚合。Has聚合不具有存在依赖性——复合对象的删除不会自动向下扩散到构件对象上。Has聚合只拥有传递性和非对称性的特点。

Has聚合的例子如图5-21所示。如果一个Trolley对象拥有多个BeerCrate对象，并且一个BeerCrate对象包含多个BeerBottle对象，那么Trolley对象就拥有这些BeerBottle对象（传递性）。如果Trolley拥有BeerCrate，那么BeerCrate就不能拥有Trolley（非对称性）。

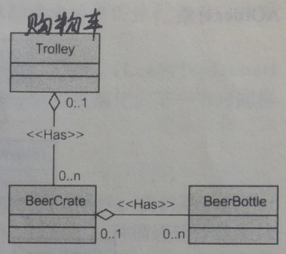


图5-21 Has聚合

iv. Member聚合

1) 考虑到聚合关系中多对多的多重性。对于存在依赖性、传递性、非对称性、固有属性（冻结特性）没有做特定的设想

2)

图5-22展示了4个Member聚合关系。一个JADSession（2.2.3.3节）对象由一个仲裁人员和一个或多个文书、用户和开发人员组成。每个构件对象能够参加一个以上的JADSession对象。

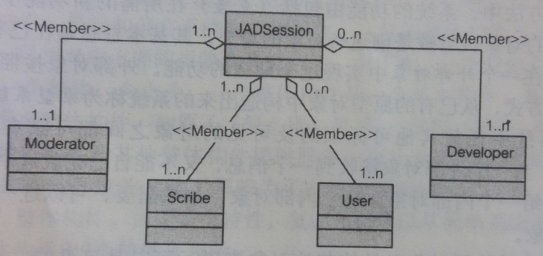


图5-22 Member聚合

b. 作为泛化的可选方案的聚合

i. 委托delegation和原型prototypical系统

1) 每当复合对象（外部对象）自身不能完成一项任务时，它能够访问它的构件对象（内部对象）中的方法，这称为委托

ii. 委托和继承

1) 聚合使用委托的方法来复用构建对象的实现

4. 高级交互(interaction)建模

- 时序图关注消息的顺序，通信图强调对象的关系
- CASE工具倾向于为顺序图的可视化提供更好的工具，通信图能在开会讨论时用作手绘的草图

a. 时序图

1) 生命线

- 一条生命线代表一个交互参与者
- 消息表示在交互的生命线之间的通信
- 接收到消息的生命线/对象激活了相关的操作方法
- 当控制流聚焦到一个对象时，在UML2.0中称为**执行规格说明**（以前叫激活）

a) 参与交互过程的类对象名称

i) 一个类的未命名实例——:Class1
ii) 一个类的命名实例——c2:Class2
iii) 一个类，即元类的一个实例——：Class（空格）3，通常显示对类自身静态方法的调用，静态方法通常也不需要实例化再调用
iv) 一个接口——:Interface1

2) 消息
   a) 同步消息synchronous messages，
      i) 调用者阻塞，即要等待响应，通常用来传递和数据
      ii) 实心箭头表示
   b) 异步消息asynchronous messages，
      i) 调用者不阻塞，允许多线程执行，通常用来传递控制
      ii) 开放箭头表示
   c) 对象创建消息object creation messages
      i) 常用关键字（new/create）命名
      ii) 开放箭头表示，例：new(a,b),a和b是提供给Class2的两个参数
   d) 回复消息reply
      i) 将交互的输出值传递给发起动作的调用者
      ii) 带有开放箭头的虚线，常标注有返回值的描述
      ◇ 注意，回复消息只是用来显示消息返回结果的两种方法中的一种。另一种是在消息语法中指明返回变量，如int=doC

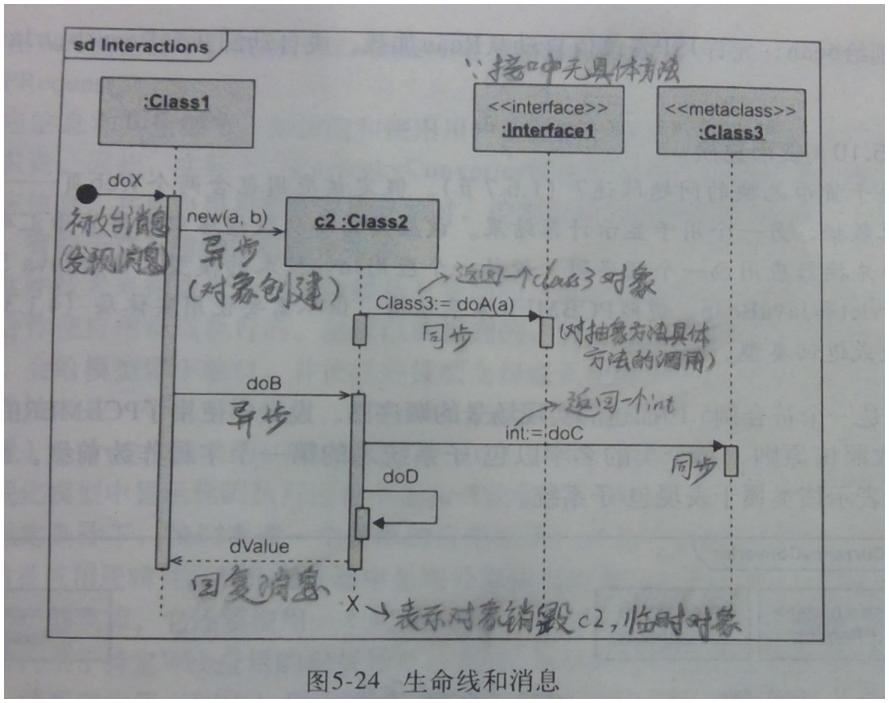


图5-24 生命线和消息

      ◇ 对象销毁在图上用X表示，在交互模型中创建的对象也将要在同一个交互模型中被销毁（如c2:Class2）对于不具备自动垃圾回收的语言（如C++）应该由模型中的另一个对象用一个单独的<<destroy>>来发起销毁。
      ◇ 在生命线代表的是接口（:Interface1）或抽象类的情况下，被调用方法是从一个实现了接口的类或一个继承了抽象类的具体类执行

3) 激活
   a) 生命线的长度表示执行时间，代表该消息调用方法的执行时间

4) 说明基本技术

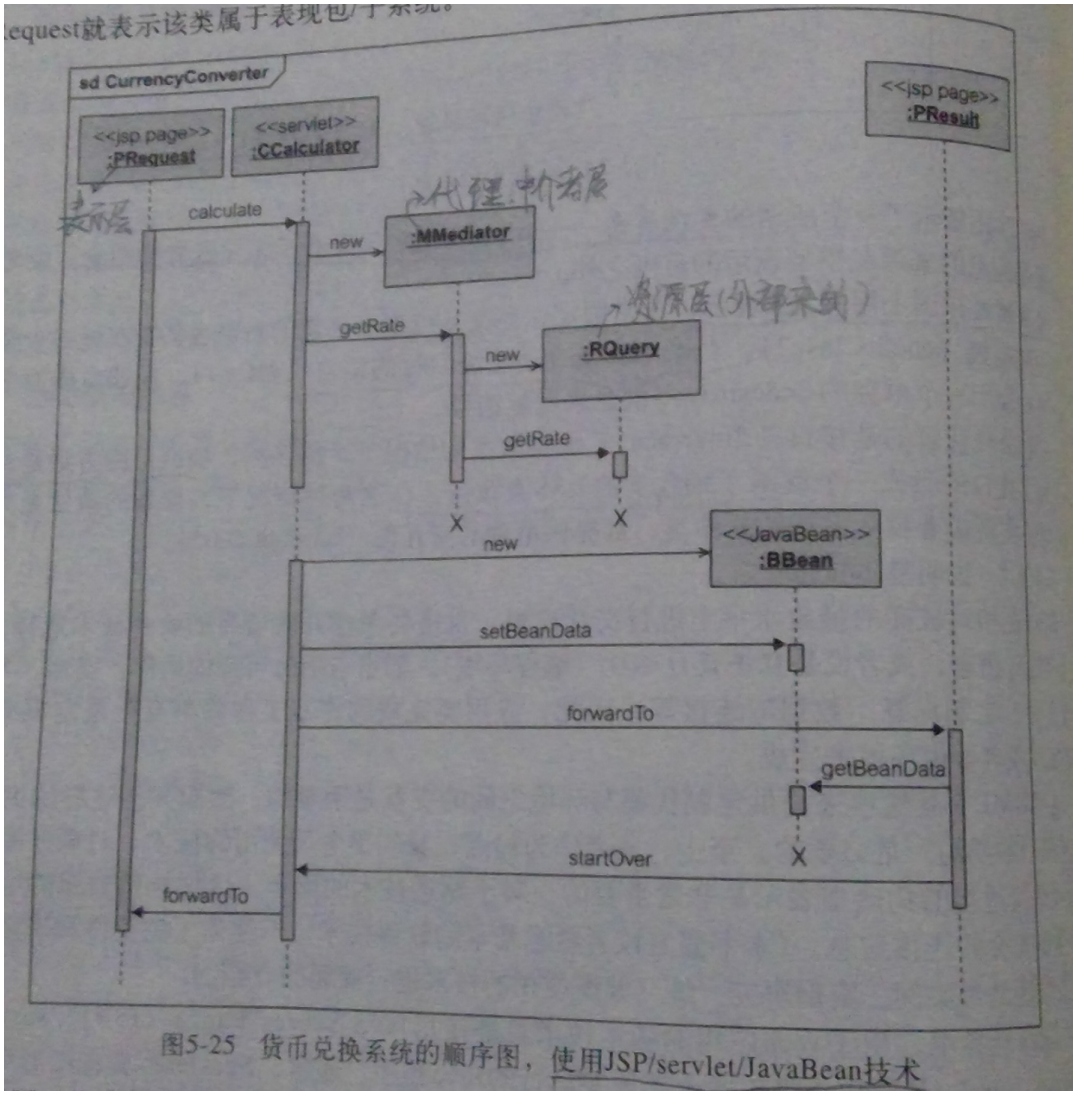


图5-25 货币兑换系统的顺序图，使用JSP/servlet/JavaBean技术

5) 片段

- 交互片段，交互可以包含更小的交互片段，称为组合片段。组合片段的语义由交互操作符确定

a) alt:可选片段，在守卫条件中表达if-then-else逻辑

b) opt:选择片段，如果守卫条件为真，执行该片段

   i) 选择片段只在可选片段的else条件内执行，并且当守卫条件为真时才执行

c) loop:循环片段，服从循环条件而重复多次的片段

d) break:中断片段，如果中断条件为真，就执行中断片段，而不执行外围片段的剩余部分

e) parallel:并行片段，允许所包含行为交替执行

f)

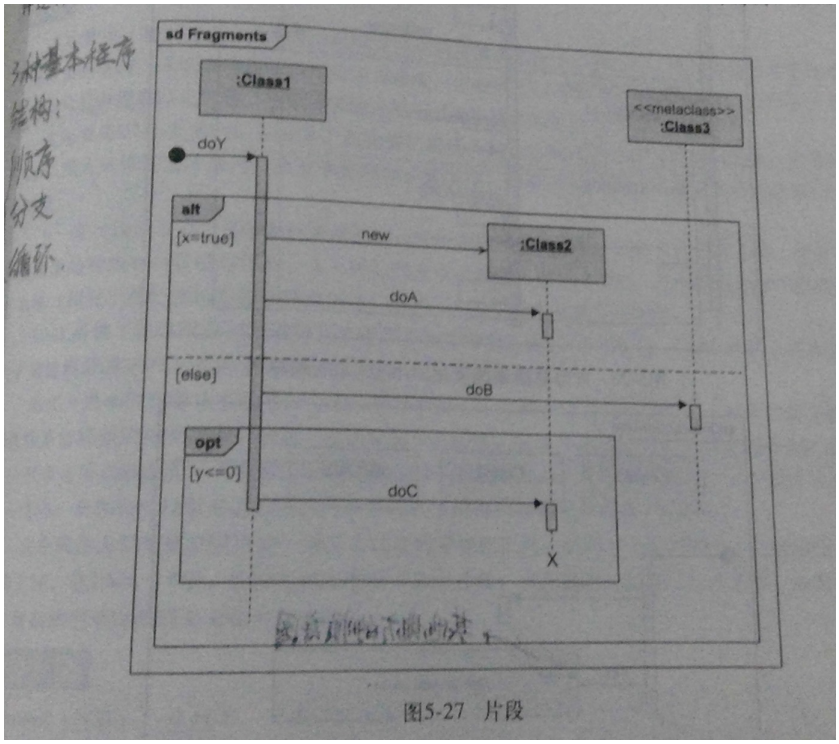


图5-27 片段

6) 交互使用

a) 一个外围交互对另一个交互的引用称为交互使用，外围交互用标签sd（顺序图）标注。

b) 交互使用标签ref（引用）标注，它指的是另一个单独创建的sd交互

c)

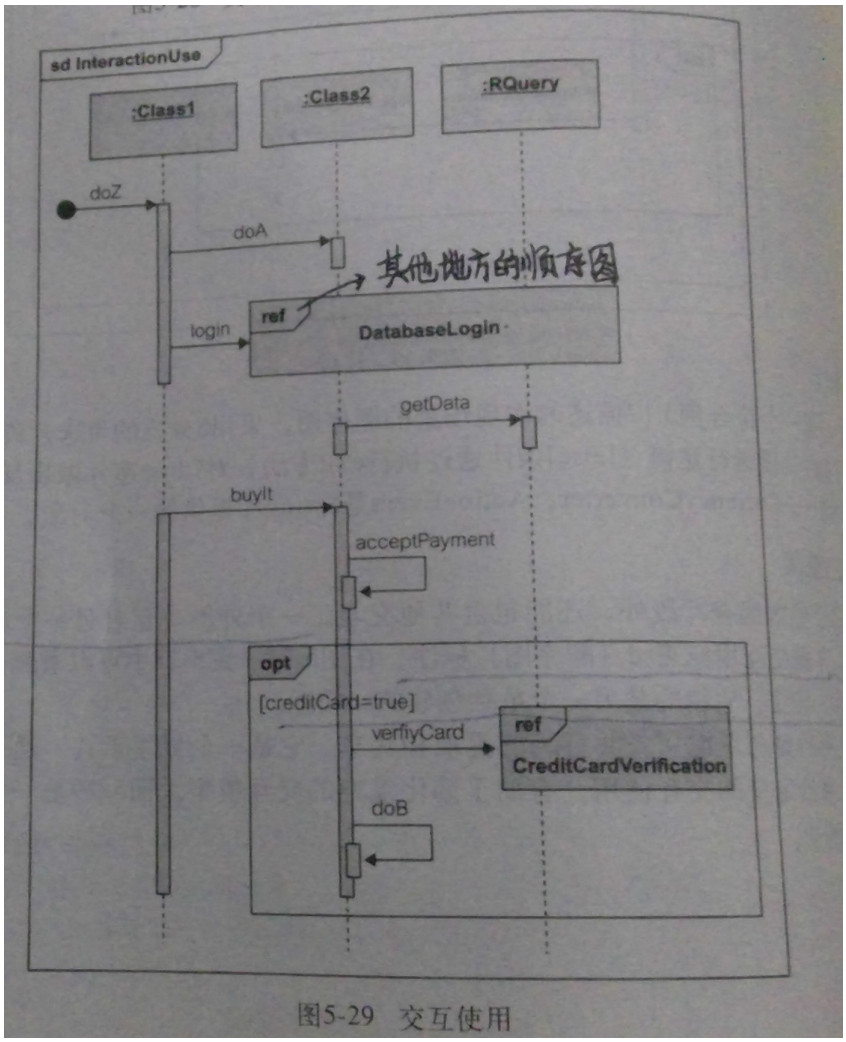


图5-29 交互使用

b. 通信图

用例之间的单向关联表示用例间的调用关系，一般情况很少，就用include，extend

表示类的对象之间的引用关系